



HACKTHEBOX

TrainingDay Writeup



2025-03-11

Prepared by: iamroot

Sherlock Author(s): Megachar0x01

Difficulty: **Easy**

Description

A reverse engineering challenge for beginners.

Scenario

A fresh new day, a well-rested cybersecurity professional—ready to learn something new after a chaotic week. Scrolling through challenges, they stumble upon TrainingDay, download the artifacts, and boom—three different binaries, each doing its own thing. Do they panic? Nope. This is exactly what they signed up for—some good old-fashioned reversing fun. If this sounds like you, what are you waiting for? Grab your debugger and dive in!

Artifacts Provided

Enter the artifacts provided along with their file hash here.

- argument_baby_1.exe - 71AA9BFA63B6F5262305030012CDD366
- argument_baby_2.exe - 0E607DABF9F4AD503A0CECFE985DE6C2
- argument_baby_3.exe - 75199A70DC118D0157C3380187C7ACEC

Skills Learnt

- Basics of reversing.
- Players will learn different calling conventions.
- basics of reversing.

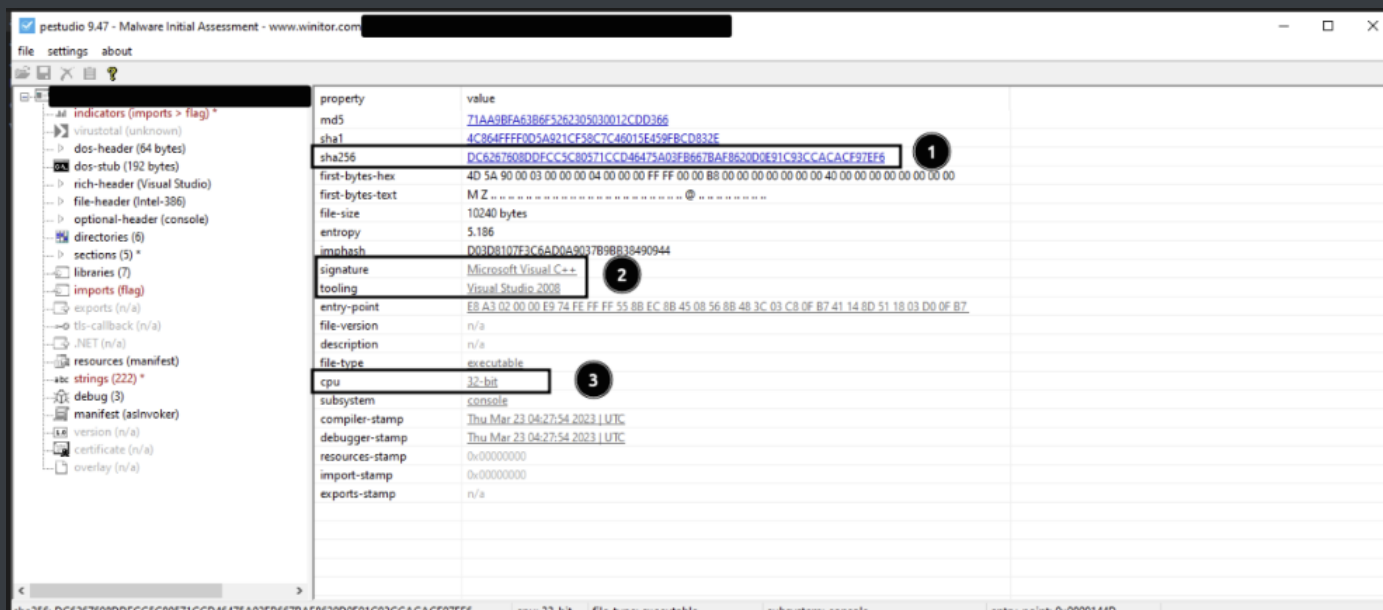
Initial Analysis

To begin the analysis, the password-protected ZIP file was unlocked using the password `hacktheblue` .

Binary Analysis

Open the binary file in Pestudio. Locate the SHA256 hash value of the file, which is a cryptographic checksum used for verifying the file's integrity and authenticity.

- Find the compiler signature, which identifies the compiler used to generate the binary file.
- Determine the CPU architecture for which the binary file was compiled.
- By following these steps, you can retrieve essential information about a binary file using Pestudio's analysis capabilities.



Locate the "File Header" tab in the Pesticide interface. Review the information presented in this tab to gain insight into the file's structure and attributes. Look for the "debug-stripped" field in the "Characteristics" section of the file header. If the value is "false," it indicates that the file contains debugging symbols, and it has not been stripped.

property	value	detail
characteristics	0x0102	
dynamic-link-library	0x0000	false
32-bit words support	0x0100	true
file-can-be-executed	0x0002	true
system-image	0x0000	false
large-address-aware	0x0000	false
debug-stripped	0x0000	false
line-stripped-from-file	0x0000	false
local-symbols-stripped-from-file	0x0000	false
relocation-stripped	0x0000	false
uniprocessor	0x0000	false
bytes-of-machine-words-reversed-Low	0x0000	false
bytes-of-machine-words-reversed-Hi	0x0000	false
media-run-from-swap	0x0000	false
network-run-from-swap	0x0000	false
general		
compiler-stamp	0x641BD54A	Thu Mar 23 04:27:54 2023 UTC
size-of-optional-header	0x00E0	224 bytes
signature	0x00004550	PE00
machine	0x014C	Intel-386
sections	0x0005	5
pointer-symbol-table	0x00000000	0x00000000
number-of-symbols	0x00000000	0x00000000

When analyzing a binary file using Pestudio, it is also possible to extract valuable information by identifying the PDB (Program Database) path. By examining the PDB path, it is often possible to determine the username of the system on which the binary file was compiled. Additionally, in some cases, the PDB path can also reveal the project name and version through the file naming scheme. Extracting this information can provide analysts with critical insights into the binary file's origins, potential security risks, and functionality.

property	value
stream (0)	
size-of-data	75 bytes
format	2 (RSDS)
first-bytes-hex	52 53 44 53 A6 0D F9 64 6C 6D EC 44 AC 36 BC AF 88 F1 2E 5E 01 00 00 00 43 3A 5C 55 73 65 7...
age	1
guid	64F90DA6-6D6C-44EC-AC36-BCAF88F12E5E
path	C:\Users\HTB\Desktop\argument\Release\argument.pdb
stamp	0x641BD54A (Thu Mar 23 04:27:54 2023 UTC)
-	save to file...
stream (1)	
size-of-data	20 bytes
format	12 (vcFeature)
first-bytes-hex	00 00 00 00 20 00 00 00 1F 00 00 00 00 00 00 1F 00 00 00
-	save to file...
stream (2)	
size-of-data	620 bytes
format	13 (PGO)
first-bytes-hex	47 43 54 4C 00 10 00 00 84 0E 00 00 2E 74 65 78 74 24 6D 6E 00 00 00 00 20 00 00 C8 00 00 00
-	save to file...

Next we will open the 'argument_baby_1.exe' binary in IDA Free and locate the main function. This is often useful for analyzing user-generated C code, as such code typically begins with a main function. This information can then be used to inform further analysis and potential remediation efforts.

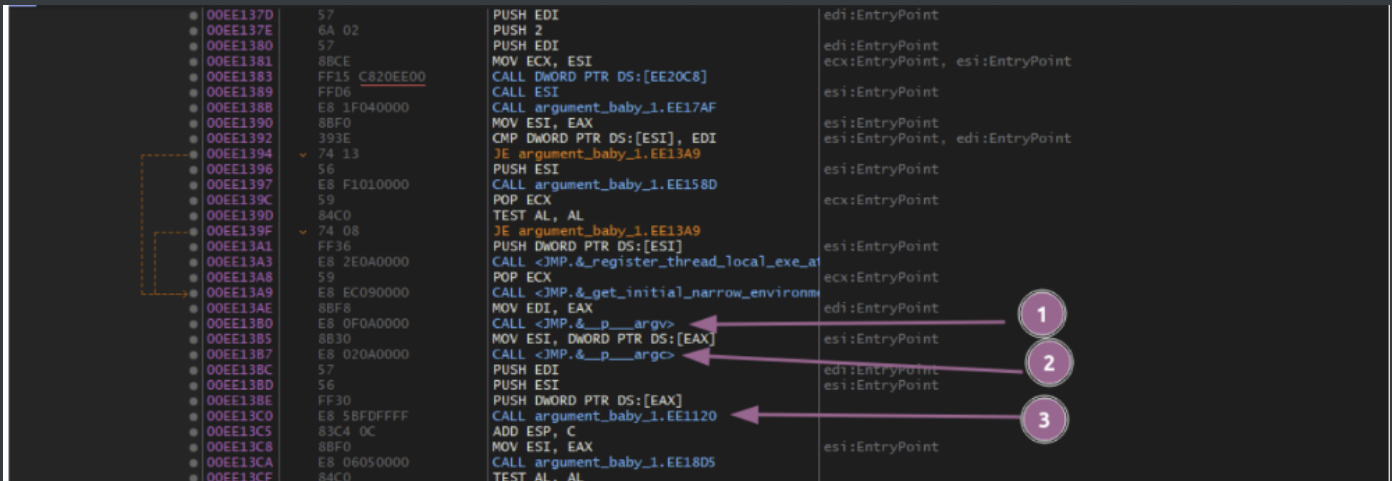
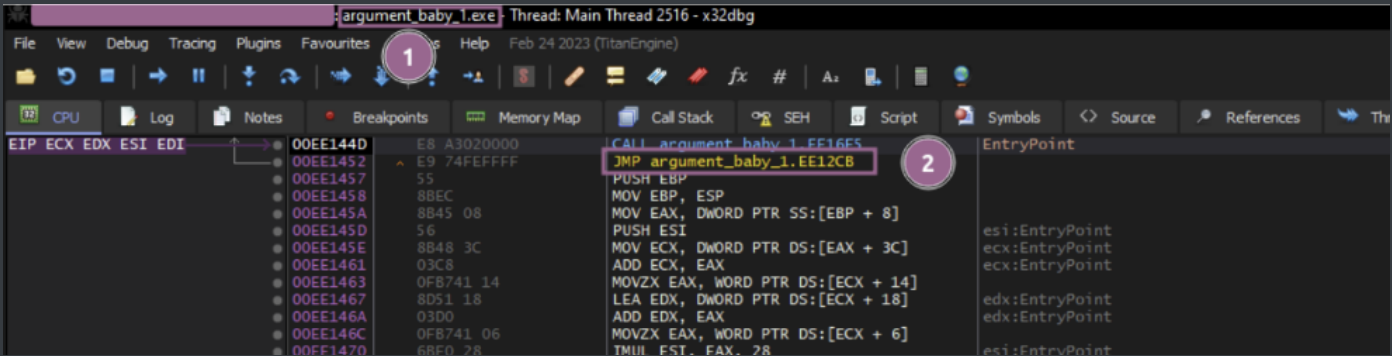
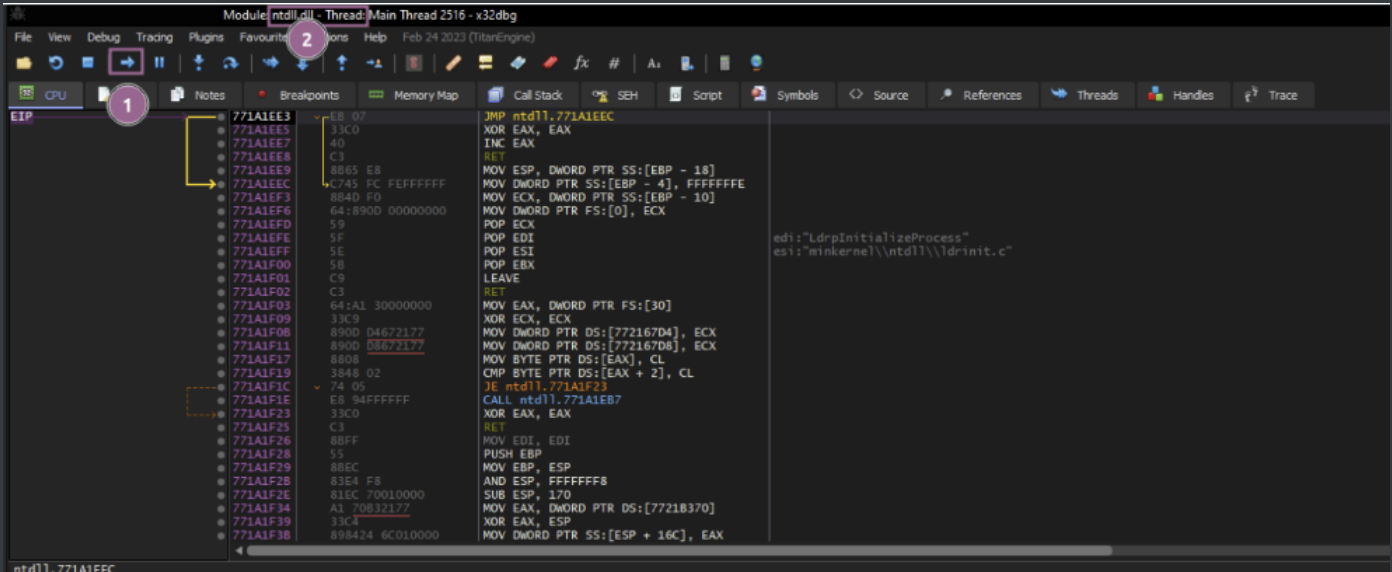
In computer programming, the stack is often used to pass arguments to functions. When a function is called using the stdcall calling convention, the caller is responsible for clearing the stack after the function has executed. If the caller does not clear the stack, the cdecl calling convention may be used instead. In this case, the called function is responsible for clearing the stack. By understanding the nuances of these calling conventions, developers can write more efficient and effective code.

```
call    sub_4010C0
add     esp, 8
mov     edx, dword ptr [ebp+var_4]
push    edx
mov     eax, dword ptr [ebp+var_8]
push    eax
mov     ecx, dword ptr [ebp+var_C]
push    ecx
mov     edx, dword ptr [ebp+var_10]
push    edx
mov     eax, dword ptr [ebp+var_14]
push    eax
call    sub_401100
add     esp, 14h
mov     [ebp+var_18], eax
mov     ecx, [ebp+var_18]
push    ecx
mov     edx, dword ptr [ebp+var_4]
push    edx
mov     eax, dword ptr [ebp+var_8]
push    eax
mov     ecx, dword ptr [ebp+var_C]
push    ecx
mov     edx, dword ptr [ebp+var_10]
push    edx
mov     eax, dword ptr [ebp+var_14]
push    eax ; char
push    offset aDDDDDD ; "%d + %d + %d + %d + %d = %d "
call    sub_401050
add     esp, 1Ch
```

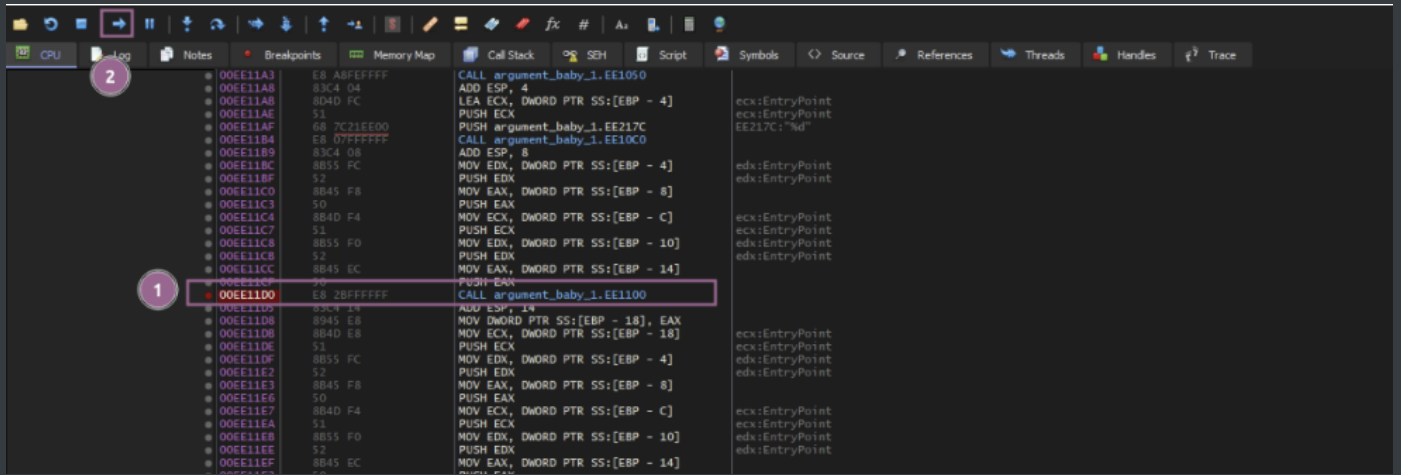
Open the 'argument_baby_1.exe' binary in xdbg . Then press Run button until top bar change from 'ntdll.dll' to 'argument_baby_1.exe'.

- Now its time to find 'main' function but right now we are at a starting point of the binary For that click on Jump Instruction to follow in debugger
 - Start scrolling until u find argv ,argc and after that the next function will be 'main'.

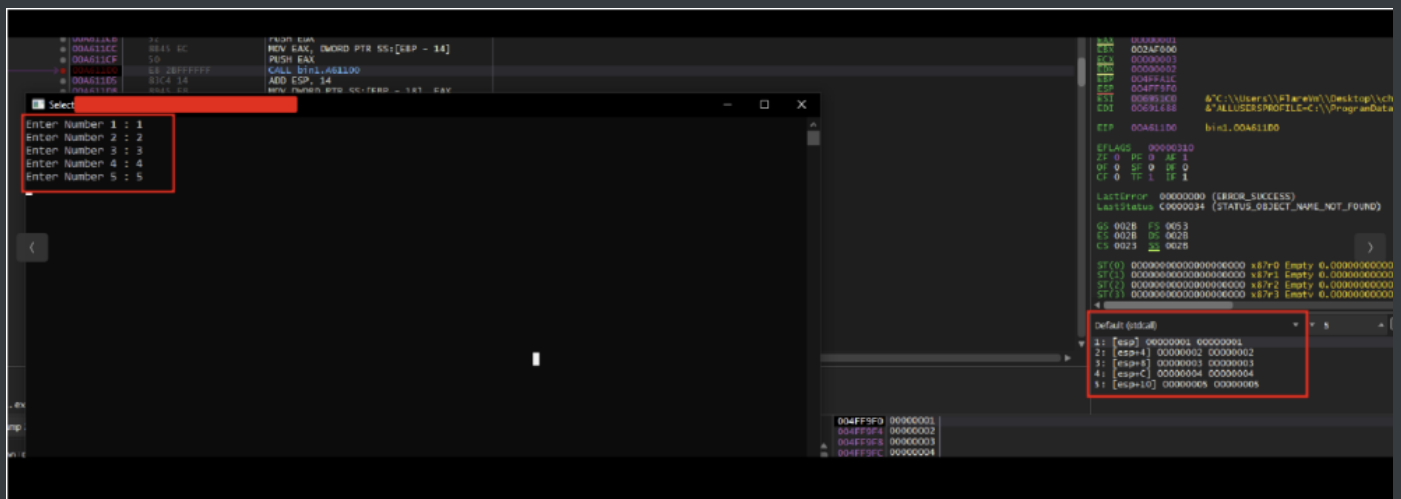
Different compiler has different implementation best way is to find function with 3 arguments after starting from start function.



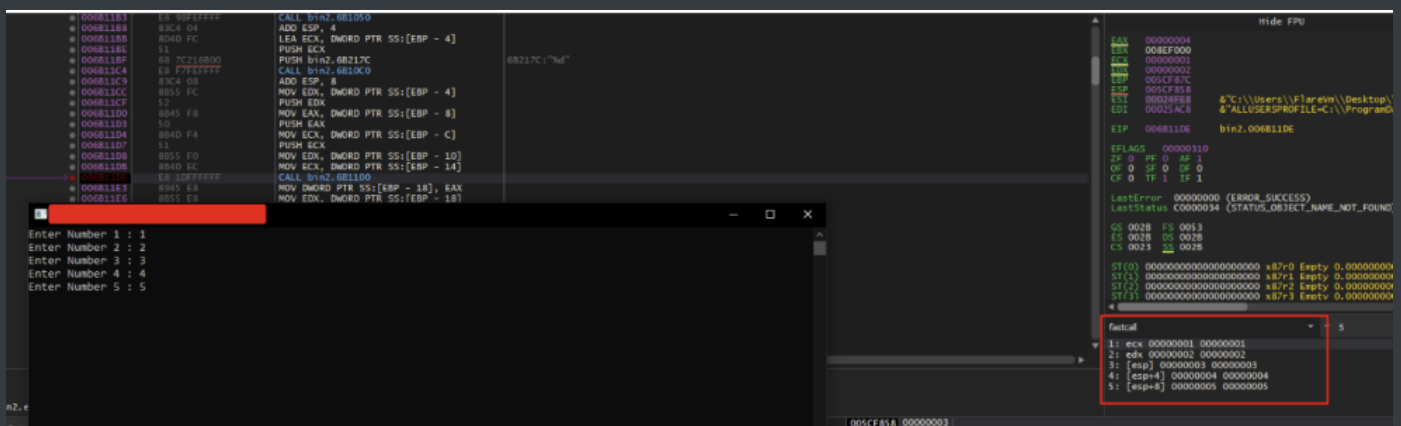
For breakpoint we click on small circle and as it turns to dark red, it means break point is set. Now press run button to get to breakpoint.



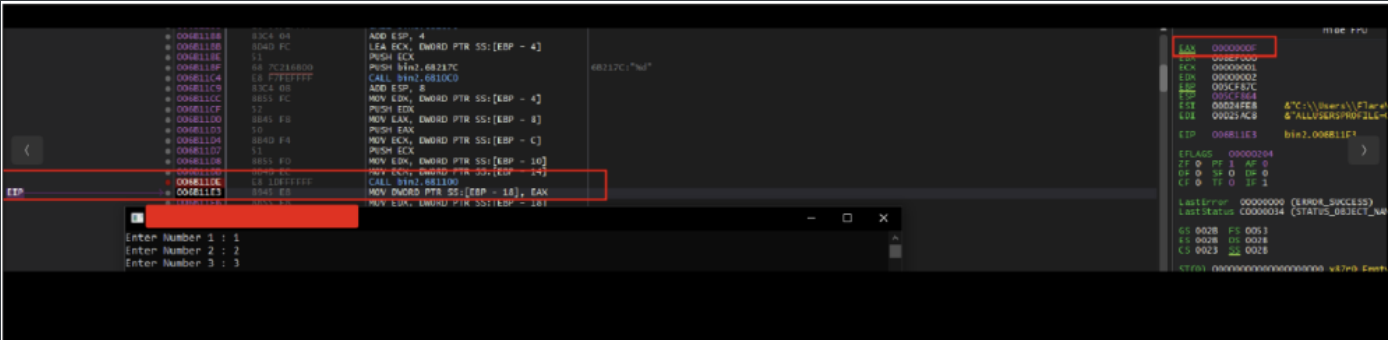
Upon running the binary, it becomes apparent that the program prompts the user for numerical input, which is then passed to a function. By examining the process by which these numbers are passed, we can gain valuable insights into the program's behavior. When using the stdcall calling convention, all arguments are typically passed through the stack



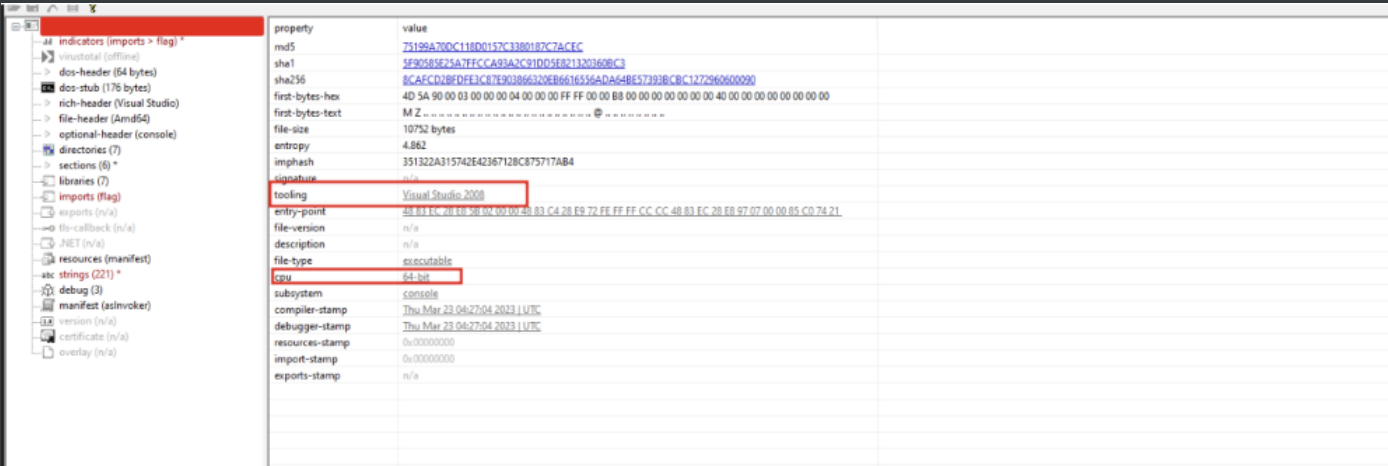
From all the knowledge we gained above we will apply it for analysing our next binary 'argument_baby_2.exe':



The calling convention of 'argument_baby_2.exe' shown in the above image is 'fastcall,' which uses both registers and the stack to pass function arguments. Typically, the first two arguments go through registers, while the rest are passed via the stack. Also in the below image you can see that the function's return value is usually stored in the EAX register, allowing the calling function or program to access it easily.



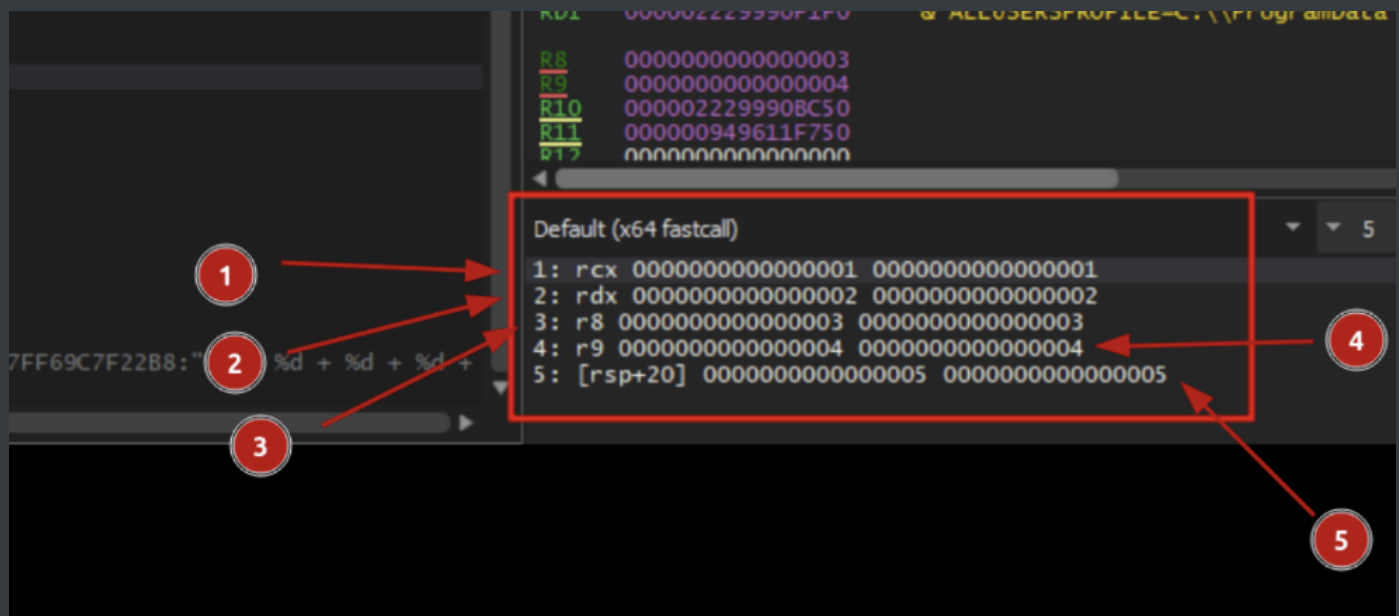
In our previous two binaries, we explored different calling conventions in binaries compiled for a 32-bit CPU architecture. However, upon analyzing the 'argument_baby_3.exe' binary in PeStudio, we discover that it is compiled for a 64-bit architecture.



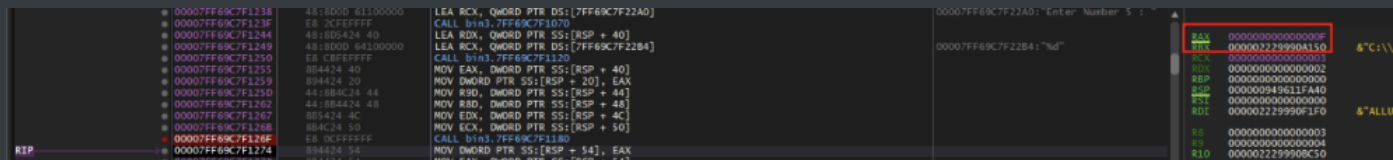
We also see that the binary is not stripped.

property	value	detail
characteristics	0x0022	
dynamic-link-library	0x0000	false
32-bit words support	0x0000	false
file-can-be-executed	0x0002	true
system-image	0x0000	false
large-address-aware	0x0020	true
debug-stripped	0x0000	false
line-stripped-from-file	0x0000	false
local-symbols-stripped-from-file	0x0000	false
relocation-stripped	0x0000	false
uniprocessor	0x0000	false
bytes-of-machine-words-reversed-Low	0x0000	false
bytes-of-machine-words-reversed-Hi	0x0000	false
media-run-from-swap	0x0000	false
network-run-from-swap	0x0000	false

As we previously learned, the fastcall calling convention utilizes both registers and the stack for passing function arguments. Typically, the first two arguments are passed through registers, while any additional arguments go through the stack. As shown in the image below, the fastcall convention remains consistent between x86 and x64 architectures.



The return value of a function is typically stored in the RAX register.



Questions

1. What is sha256sum hash of argument_baby_1.exe ?

Answer: `dc6267608ddfcc5c80571ccd46475a03fb667baf8620d0e91c93ccacacf97ef6`

2. Can you find development enviroment of malware author (e.g:ide) ?

Answer: `visual studio`

3. What CPU architecture was argument_baby_1.exe compiled for ?

Answer: `32 bit`

4. Is argument_baby_1.exe a symbol-stripped binary?

Answer: `false`

5. Retrieve the full PDB file path from the debug information of argument_baby_1.exe

Answer: `C:\Users\HTB\Desktop\argument\Release\argument.pdb`

6. The malware author was using a Windows OS. What was the associated username?

Answer: `HTB`

7. What calling convention is used in argument_baby_1.exe ?

Answer: `cdecl`

8. How is the 1st argument passed to the function in argument_baby_1.exe?

Answer: `esp`

9. How is the 2nd argument passed to the function in argument_baby_1.exe?

Answer: `esp+4`

10. How is the 3rd argument passed to the function in argument_baby_1.exe?

Answer: `esp+8`

11. How is the 4th argument passed to the function in argument_baby_1.exe?

Answer: `esp+c`

12. How is the 5th argument passed to the function in argument_baby_1.exe?

Answer: `esp+10`

13. Which CPU register holds the function's return value in argument_baby_1.exe?

Answer: `eax`

14. What calling convention is used in argument_baby_2.exe ?

Answer: `fastcall`

15. How is the 1st argument passed to the function in argument_baby_2.exe?

Answer: `ecx`

16. How is the 2nd argument passed to the function in argument_baby_2.exe?

Answer: `edx`

17. How is the 3rd argument passed to the function in argument_baby_2.exe?

Answer: `esp`

18. How is the 4th argument passed to the function in argument_baby_2.exe?

Answer: `esp+4`

19. How is the 5th argument passed to the function in argument_baby_2.exe?

Answer: `esp+8`

20. What CPU architecture was argument_baby_3.exe compiled for ?

Answer: 64 bit

21. How is the 1st argument passed to the function in argument_baby_3.exe?

Answer: rcx

22. How is the 2nd argument passed to the function in argument_baby_3.exe?

Answer: rdx

23. How is the 3rd argument passed to the function in argument_baby_3.exe?

Answer: r8

24. How is the 4th argument passed to the function in argument_baby_3.exe?

Answer: r9

25. How is the 5th argument passed to the function in argument_baby_3.exe?

Answer: rsp+20

26. In the case of argument_baby_3.exe, which CPU register stores the function's return value?

Answer: rax